UNIVERSITÉ MOHAMMED PREMIER
Faculté des Sciences d'Oujda — Département Informatique

Programmation Système
en Langage C sous Linux

Polycopié de Cours

Pr. Karima AISSAOUI
k.aissaoui@ump.ac.ma

Avant-propos

Ce polycopié est destiné aux étudiants de licence qui suivent le cours de Programmation Système en langage C sous Linux. Il constitue un support pédagogique structuré couvrant les thèmes fondamentaux du programme, depuis la gestion des processus jusqu'à la communication inter-processus.

Chaque chapitre présente les concepts théoriques, les syntaxes des appels système et des fonctions de bibliothèque, ainsi que des exemples de programmes complets avec leurs résultats d'exécution. Les exemples ont été conçus pour être compilés et exécutés directement sur un système Linux (Ubuntu, Debian, Mint ou équivalent).

Ce document s'appuie principalement sur la norme POSIX.1-2017 et les travaux de référence de la communauté Unix/Linux, notamment :

- W. Richard Stevens, « Advanced Programming in the UNIX Environment » — la référence incontournable de la programmation système Unix/Linux.
- Michael Kerrisk, « The Linux Programming Interface » — guide exhaustif des appels système Linux modernes.
- Brian W. Kernighan & Dennis M. Ritchie, « The C Programming Language » — bible du langage C.

Les étudiants sont encouragés à consulter les pages de manuel (man) sous Linux pour toute précision sur les signatures des fonctions, les codes d'erreur et les comportements spécifiques à chaque version du noyau.

Code C
```
$ man 2 fork        # Section 2 = appels système
$ man 3 pthread_create  # Section 3 = fonctions de bibliothèque
$ man 7 signal      # Section 7 = concepts généraux
```

Table des matières

Avant-propos 2

Chapitre 1 — Introduction à la programmation système 4
- 1.1 Rôle du système d'exploitation et du noyau Linux
- 1.2 Appels système et bibliothèque C
- 1.3 Outils de développement
- 1.4 Langages compilés vs interprétés
- 1.5 Premier programme C sous Linux

Chapitre 2 — Gestion des processus 9
- 2.1 Programme et processus — définitions
- 2.2 Contexte d'un processus : le PCB
- 2.3 États d'un processus et transitions
- 2.4 Hiérarchie et identifiants (PID, PPID, UID, GID)
- 2.5 Création d'un processus — fork()
- 2.6 Terminaison — exit(), wait(), waitpid()
- 2.7 Remplacement d'image — famille exec()
- 2.8 La primitive sleep()
- 2.9 Gestion des erreurs — errno et perror()

Chapitre 3 — Gestion des threads POSIX 18
- 3.1 Processus lourd vs processus léger
- 3.2 Création d'un thread — pthread_create()
- 3.3 Terminaison — pthread_exit() et pthread_join()
- 3.4 Détachement d'un thread — pthread_detach()
- 3.5 Variables partagées et section critique
- 3.6 Exclusion mutuelle — mutex
- 3.7 Variables de condition

Chapitre 4 — Gestion des signaux 30
- 4.1 Notion de signal
- 4.2 États d'un signal
- 4.3 Tableau des signaux courants
- 4.4 Envoyer un signal — kill()
- 4.5 Installer un gestionnaire — signal()
- 4.6 Signaux non interceptables

Chapitre 5 — Communication inter-processus : les tubes 38
- 5.1 Tubes anonymes — pipe()
- 5.2 Écriture et lecture dans un tube
- 5.3 Tubes nommés (FIFO) — mkfifo()

5.4 Droits d'accès
5.5 Ouverture bloquante et non-bloquante

Références bibliographiques 50

Chapitre 1 — Introduction à la programmation système

1.1 Rôle du système d'exploitation

Définition — Système d'exploitation (SE) : *Logiciel spécialisé qui sert d'intermédiaire entre le matériel (CPU, RAM, périphériques) et les applications utilisateur, en gérant l'allocation des ressources et en fournissant une interface de programmation standardisée.*

On distingue deux grandes catégories de programmes :

- **Programmes d'application :** logiciels utilisateur développés en C, Python, Java, etc. Ils s'appuient sur le SE pour accéder aux ressources.
- **Programmes systèmes :** composants du SE lui-même : gestionnaire de fichiers, ordonnanceur, pilotes de périphériques, gestionnaire de mémoire, pile réseau.

Le langage C a été inventé par Dennis Ritchie aux Bell Labs (1972) précisément pour réécrire le noyau Unix en un langage portable et efficace. C'est pourquoi C reste le langage dominant pour la programmation système à ce jour.

Référence : *Kernighan & Ritchie, « The C Programming Language »*[1] *; Stevens & Rago, « Advanced Programming in the UNIX Environment »*[2]

1.2 Architecture du noyau Linux

Le noyau Linux est organisé autour de trois sous-systèmes fondamentaux :

- **Gestion des processus :** création, ordonnancement, terminaison et communication entre processus.
- **Gestion de la mémoire :** allocation de la mémoire virtuelle et physique, pagination, segmentation.
- **Gestion des E/S :** abstraction des périphériques via le système de fichiers, les pilotes et les mécanismes d'interruption.

Définition — Appel système (syscall) : *Interface fondamentale entre un programme utilisateur et le noyau Linux. Un appel système suspend l'exécution en mode utilisateur, bascule le CPU en mode noyau, exécute la fonction demandée, puis retourne le résultat au programme.*

En langage C, les appels système sont accessibles directement ou via des fonctions d'enveloppe (wrappers) fournies par la bibliothèque C standard (glibc). Par exemple, fork() est une fonction glibc qui déclenche l'appel système clone() du noyau.

Référence : *Kerrisk, « The Linux Programming Interface »*[3] *; Documentation officielle du noyau Linux*[4]

---

[1] Brian W. Kernighan, Dennis M. Ritchie, "The C Programming Language", 2nd ed., Prentice Hall, 1988.
[2] W. Richard Stevens, Stephen A. Rago, "Advanced Programming in the UNIX Environment", 3rd ed., Addison-Wesley, 2013.
[3] Michael Kerrisk, "The Linux Programming Interface", No Starch Press, 2010. (Référence complète des appels système Linux.)
[4] Linus Torvalds et al. — Documentation officielle du noyau Linux : https://www.kernel.org/doc/html/latest/

1.3 Outils de développement C sous Linux

La chaîne de compilation C sous Linux s'articule autour de cinq catégories d'outils :

- **Éditeurs de texte :** vi/vim, nano, gedit, VS Code. Vi est disponible sur tous les systèmes Unix.
- **Compilateur :** GCC (GNU Compiler Collection) est le standard. Clang/LLVM est une alternative moderne.
- **Éditeur de liens :** ld (GNU linker), intégré à GCC. Résout les symboles externes et produit l'exécutable.
- **Débogueur :** GDB (GNU Debugger) permet l'exécution pas à pas, l'inspection des variables et de la pile.
- **Utilitaires :** make (automatisation), valgrind (fuites mémoire), strace (trace des appels système), nm (symboles).

Commandes vi essentielles

Code C

```
$ vi mon_programme.c   # créer/éditer un fichier

Mode insertion    → touche  i
Retour mode normal→ touche  Echap
:w                → sauvegarder
:q!               → quitter sans sauvegarder
:wq! ou :x        → sauvegarder et quitter
dd                → supprimer la ligne courante
ndd               → supprimer n lignes
x                 → effacer le caractère sous le curseur
/mot              → rechercher 'mot'
:%s/ancien/nouveau/g → remplacer dans tout le fichier
```

1.4 Langages compilés vs interprétés

La distinction entre langages compilés et interprétés a des implications directes sur les performances et la portabilité des programmes systèmes.

Critère	Compilé (C, C++)	Interprété (Python, Bash)
Exécution	Directe (binaire natif)	Via interpréteur
Vitesse	Rapide — optimisé pour le CPU	Plus lente
Portabilité	Recompiler par plateforme	Multi-plateforme
Débogage	Nécessite GDB	Messages d'erreur immédiats
Usage système	Idéal (OS, drivers, serveurs)	Scripts, automatisation

1.5 Compiler et exécuter un programme C

La commande gcc traduit le code source C en un exécutable en plusieurs phases internes : prétraitement, compilation, assemblage, édition de liens.

Synopsis

```
gcc [options] fichier.c -o executable

Options fréquentes :
  -Wall       activer tous les avertissements
  -Wextra     avertissements supplémentaires
  -g          inclure les symboles de débogage (pour gdb)
  -O2         optimisation niveau 2
  -lpthread   lier la bibliothèque POSIX threads
```

```
    -lm          lier la bibliothèque mathématique
```

Exemple — premier programme complet

Code C

```c
/* bonjour.c — premier programme Linux */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    printf("Bonjour, Linux !\n");
    printf("Mon PID = %d\n", getpid());
    return EXIT_SUCCESS;
}
```

Résultat d'exécution

```
$ gcc -Wall -o bonjour bonjour.c
$ ./bonjour
Bonjour, Linux !
Mon PID = 7842
```

⚠ **Note :** EXIT_SUCCESS (valeur 0) et EXIT_FAILURE (valeur 1) sont définis dans <stdlib.h> — préférez-les aux constantes numériques brutes.

Chapitre 2 — Gestion des processus

Référence du chapitre : *Stevens & Rago, chap. 7–10* [5] *; Kerrisk, chap. 24–28* [6] *; man 2 fork, man 2 execve, man 2 waitpid*[7]

2.1 Programme et processus — définitions

Définition — Programme : *Fichier exécutable stocké sur disque (section .text, .data, .bss). Entité passive.*

Définition — Processus : *Programme en cours d'exécution. Entité active qui possède son propre espace mémoire virtuel, ses descripteurs de fichiers, son compteur ordinal, ses registres CPU et son état.*

Linux est un système multi-tâche préemptif : le noyau découpe le temps CPU en tranches (quantum, de l'ordre de la milliseconde) et les répartit entre les processus prêts. Du point de vue de l'utilisateur, les processus semblent s'exécuter simultanément (parallélisme apparent sur mono-processeur, réel sur multi-cœurs).

⚠ **Note :** Sur un système mono-processeur, il n'y a qu'un seul processus actif à la fois. Le parallélisme observé est le résultat de la commutation rapide entre processus.

2.2 Contexte d'un processus — PCB

Pour gérer les processus, le noyau maintient une table des processus. Chaque entrée est un Bloc de Contrôle de Processus (PCB — Process Control Block) qui contient toutes les informations nécessaires pour suspendre et reprendre l'exécution d'un processus.

Champ du PCB	Description
PID / PPID	Identifiant unique du processus et de son père
UID / GID	Identifiant de l'utilisateur et du groupe propriétaire
État	Actif, Prêt, Bloqué ou Terminé
Contexte CPU	Valeur de tous les registres au moment de la suspension
Table des fichiers	Descripteurs de fichiers ouverts (0=stdin, 1=stdout, 2=stderr…)
Répertoire courant	Chemin du répertoire de travail du processus
Espace mémoire	Pages de code (.text), données (.data/.bss), pile, tas
Signaux	Masque des signaux bloqués et signaux en attente
Terminal attaché	Tty ou pts depuis lequel le processus a été lancé

[7] Pages de manuel Linux (man pages) : man fork, man exec, man signal, man pthread_create, man pipe, man mkfifo — disponibles via la commande man sous Linux.

L'opération de sauvegarde du PCB courant et de restauration d'un autre s'appelle commutation de contexte (context switch). Elle est entièrement gérée par le noyau et est transparente pour les programmes utilisateur.

2.3 États d'un processus et transitions

- **Actif (Élu) :** le processus s'exécute sur le CPU. Sur un mono-processeur, un seul processus est actif à la fois.
- **Prêt :** le processus est en mémoire et attend que le CPU lui soit attribué par l'ordonnanceur.
- **Bloqué (En attente) :** le processus attend un événement extérieur (entrée clavier, fin d'E/S, signal…). Il ne consomme pas de CPU.
- **Terminé (Zombie) :** le processus a terminé son exécution mais son PCB reste en mémoire jusqu'à ce que le père appelle wait(). Un zombie qui n'est jamais attendu reste en mémoire indéfiniment — il faut toujours appeler wait().

Transitions principales : Actif → Prêt (quantum expiré), Actif → Bloqué (attente ressource), Bloqué → Prêt (ressource disponible), Prêt → Actif (ordonnanceur choisit le processus).

2.4 Hiérarchie et identifiants

Tous les processus Linux forment un arbre enraciné en init (PID=1) ou systemd. Chaque processus (sauf init) a un père dont il hérite l'environnement.

Synopsis

```
#include <unistd.h>
#include <sys/types.h>

pid_t getpid(void);     /* PID du processus courant */
pid_t getppid(void);    /* PID du processus père    */
uid_t getuid(void);     /* UID de l'utilisateur     */
gid_t getgid(void);     /* GID du groupe            */
```

Code C

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("PID  = %d\n", (int)getpid());
    printf("PPID = %d\n", (int)getppid());
    printf("UID  = %d\n", (int)getuid());
    printf("GID  = %d\n", (int)getgid());
    return 0;
}
```

Résultat d'exécution

```
PID  = 5201
PPID = 4830
UID  = 1000
GID  = 1000
```

La commande shell ps aux affiche l'ensemble des processus du système. La commande pstree visualise l'arbre de parenté.

Code C

```
$ ps aux | head -5
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT  COMMAND
root         1  0.0  0.1  22560  4356 ?        Ss   /sbin/init
root         2  0.0  0.0      0     0 ?        S    [kthreadd]

$ pstree -p   # afficher l'arbre avec PIDs
```

2.5 Création d'un processus — fork()

Définition — fork() : *Appel système qui duplique le processus appelant. Les deux processus (père et fils) reprennent l'exécution immédiatement après fork(), avec des espaces mémoire indépendants (Copy-On-Write).*

Synopsis

```
#include <unistd.h>
#include <sys/types.h>

pid_t fork(void);

Retour : PID du fils  dans le processus père
          0            dans le processus fils
         -1            en cas d'erreur (errno est positionné)
```

Après fork(), les deux processus partagent le même code mais chacun possède une copie privée de ses données. Le mécanisme Copy-On-Write (COW) du noyau fait en sorte que la duplication effective d'une page mémoire n'a lieu qu'au moment où l'un des deux processus tente de la modifier, ce qui rend fork() très efficace.

Exemple 1 — identification père/fils

Code C

```c
/* fork1.c */
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        /* --- CODE DU FILS --- */
        printf("[FILS]  PID=%d  PPID=%d\n", getpid(), getppid());
    } else {
        /* --- CODE DU PERE --- */
        printf("[PERE]  PID=%d  PID_fils=%d\n", getpid(), pid);
    }
    return 0;
}
```

Résultat d'exécution

```
[PERE]  PID=3100  PID_fils=3101
[FILS]  PID=3101  PPID=3100
(L'ordre d'affichage peut varier)
```

Exemple 2 — indépendance des variables

Après fork(), le fils hérite d'une copie des variables du père, mais les modifications dans l'un n'affectent pas l'autre.

Code C

```c
/* fork2.c */
#include <stdio.h>
#include <unistd.h>

int main(void) {
    int n = 100;
```

```
    printf("Avant fork : n = %d\n", n);
    pid_t pid = fork();

    if (pid == 0) {
        n = n * 2;      /* modification dans le FILS */
        printf("Fils  : n = %d (apres *2)\n", n);
    } else {
        n = n - 50;     /* modification dans le PERE */
        printf("Pere  : n = %d (apres -50)\n", n);
        wait(NULL);
    }
    return 0;
}
```

Résultat d'exécution

```
Avant fork : n = 100
Pere  : n = 50 (apres -50)
Fils  : n = 200 (apres *2)
```

⚠ **Note :** Les deux processus ont démarré avec n=100 mais opèrent chacun sur leur propre copie mémoire.

Exemple 3 — arbre de processus

Il est possible de créer plusieurs fils successifs. Attention à bien appeler exit() dans chaque fils pour éviter que le fils lui-même appelle fork() lors du tour suivant.

Code C

```
/* fork3.c — créer 3 fils indépendants */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main(void) {
    int i;
    pid_t pid;

    for (i = 1; i <= 3; i++) {
        pid = fork();
        if (pid == 0) {
            printf("[Fils %d] PID=%d, PPID=%d\n",
                   i, getpid(), getppid());
            exit(0);         /* le fils ne doit pas boucler */
        } else if (pid < 0) {
            perror("fork"); exit(1);
        }
    }
    /* Le père attend la fin de tous ses fils */
    while (wait(NULL) > 0);
    printf("[Pere] tous les fils ont termine.\n");
    return 0;
}
```

Résultat d'exécution

```
[Fils 1] PID=4010, PPID=4009
[Fils 2] PID=4011, PPID=4009
[Fils 3] PID=4012, PPID=4009
[Pere] tous les fils ont termine.
```

2.6 Terminaison — exit(), wait(), waitpid()

Un processus se termine normalement par return depuis main() ou par appel explicite de exit(). Il peut aussi être tué par un signal (SIGKILL, SIGTERM). La valeur de retour (status code) est transmise au processus père.

exit()

Synopsis

```
#include <stdlib.h>

void exit(int status);    /* ne retourne jamais */

/* Constantes standard */
#define EXIT_SUCCESS  0
#define EXIT_FAILURE  1
```

exit() exécute les fonctions enregistrées par atexit(), vide les tampons des flux stdio, puis appelle _exit() (appel système kernel) pour libérer les ressources du processus.

wait() et waitpid()

Synopsis

```
#include <sys/wait.h>

pid_t wait(int *statut);
/* Attend la fin d'UN fils quelconque. Retourne son PID. */

pid_t waitpid(pid_t pid, int *statut, int options);
/* pid=-1 : attendre n'importe quel fils (comme wait) */
/* pid>0  : attendre le fils de PID pid              */
/* options=WNOHANG : non bloquant (retourne 0 si pas terminé) */
```

Macros d'analyse du statut retourné :

- WIFEXITED(statut) — vrai si le fils a terminé normalement
- WEXITSTATUS(statut) — code de retour du fils (si WIFEXITED)
- WIFSIGNALED(statut) — vrai si le fils a été tué par un signal
- WTERMSIG(statut) — numéro du signal tueur (si WIFSIGNALED)

Code C

```
/* wait_demo.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int statut;
    pid_t pid = fork();

    if (pid == 0) {
        printf("Fils : calcul en cours...\n");
        sleep(2);
        exit(42);
    }
    printf("Pere : attente du fils PID=%d\n", pid);
    waitpid(pid, &statut, 0);

    if (WIFEXITED(statut))
        printf("Pere : fils termine normalement, code=%d\n",
               WEXITSTATUS(statut));
    else if (WIFSIGNALED(statut))
        printf("Pere : fils tue par signal %d\n",
               WTERMSIG(statut));
    return 0;
}
```

Résultat d'exécution

```
Pere : attente du fils PID=6010
Fils : calcul en cours...
Pere : fils termine normalement, code=42
```

⚠ **Note :** Un processus terminé dont le père n'a pas appelé wait() devient un zombie. Il occupe une entrée dans la table des processus. Trop de zombies épuisent les ressources du noyau.

2.7 Remplacement d'image — famille exec()

La famille exec() remplace l'image du processus courant (code, données, pile) par un nouveau programme. Le PID reste identique. Aucun nouveau processus n'est créé.

Synopsis

```
#include <unistd.h>

int execl (const char *path, const char *arg, ..., NULL);
int execv (const char *path, char *const argv[]);
int execlp(const char *file, const char *arg, ..., NULL);
int execvp(const char *file, char *const argv[]);
int execle(const char *path, const char *arg, ..., NULL,
           char *const envp[]);

/* Retour : -1 en cas d'erreur (exec réussi ne retourne jamais) */

Mémotechnique :
  l = liste  d'arguments (arg0, arg1, ..., NULL)
  v = vecteur d'arguments (tableau argv[])
  p = recherche dans PATH
  e = environnement explicite (envp[])
```

Code C

```c
/* exec_demo.c — fork + exec pour lancer 'ls -lh' */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        /* Le fils remplace son image par la commande ls */
        execlp("ls", "ls", "-lh", NULL);
        /* Si on arrive ici, exec a échoué */
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    wait(NULL);
    printf("[Pere] listage termine.\n");
    return 0;
}
```

Résultat d'exécution

```
-rwxr-xr-x 1 user user  12K mars 10 exec_demo
-rw-r--r-- 1 user user 1.2K mars 10 exec_demo.c
[Pere] listage termine.
```

2.8 La primitive sleep()

Synopsis

```
#include <unistd.h>

unsigned int sleep(unsigned int secondes);
/* Retour : 0 si le délai s'est écoulé entièrement,
            nombre de secondes restantes si interrompu par un signal */
```

sleep() suspend le processus appelant pendant le nombre de secondes spécifié. Elle peut être interrompue par un signal, auquel cas elle retourne le nombre de secondes restantes.

Code C

```
/* sleep_demo.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Fils  : je dors 2s...\n");
        sleep(2);
        printf("Fils  : reveil !\n");
        return 0;
    }
    printf("Pere  : je dors 5s...\n");
    sleep(5);
    printf("Pere  : reveil !\n");
    wait(NULL);
    return 0;
}
```

Résultat d'exécution

```
Pere  : je dors 5s...
Fils  : je dors 2s...
Fils  : reveil !
Pere  : reveil !
```

2.9 Gestion des erreurs — errno et perror()

En cas d'erreur, les appels système retournent -1 et positionnent la variable globale errno (déclarée dans <errno.h>) avec un code identifiant la cause de l'erreur.

Synopsis

```
#include <errno.h>    /* variable errno         */
#include <stdio.h>    /* fonction perror()      */
#include <string.h>   /* fonction strerror()    */

void perror(const char *prefixe);
/* Affiche : prefixe: description_de_l_erreur\n */

char *strerror(int errnum);
/* Retourne la chaîne décrivant le code d'erreur errnum */
```

Code C

```
/* erreur_demo.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

int main(void) {
```

```
    pid_t pid = fork();
    if (pid == -1) {
        fprintf(stderr, "Erreur fork : %s (errno=%d)\n",
                strerror(errno), errno);
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {
        /* Tenter d'ouvrir un fichier inexistant */
        if (access("/fichier_inexistant", R_OK) == -1)
            perror("access");
        exit(0);
    }
    wait(NULL);
    return 0;
}
```

Résultat d'exécution

```
access: No such file or directory
```

⚠ **Note :** Appelez perror() ou lisez errno immédiatement après l'appel en erreur : un appel système réussi peut remettre errno à 0.

Chapitre 3 — Gestion des threads POSIX

Référence du chapitre : *Butenhof, « Programming with POSIX Threads »*[8] *; Kerrisk, chap. 29–33* [9] *; norme POSIX.1-2017*[10]

3.1 Processus lourd vs processus léger

Définition — Thread (processus léger) : *Unité d'exécution au sein d'un processus. Les threads d'un même processus partagent l'espace d'adressage, le code, les données globales et les descripteurs de fichiers. Chaque thread possède cependant sa propre pile d'exécution, ses propres registres et son identifiant de thread (pthread_t).*

Avantages des threads par rapport aux processus :

- **Coût réduit :** la commutation de contexte inter-thread est moins coûteuse car il n'y a pas de changement d'espace d'adressage.
- **Communication directe :** les threads communiquent via les variables partagées sans mécanisme IPC.
- **Réactivité :** une application peut continuer à répondre (ex. interface graphique) pendant qu'un thread effectue un traitement long.

Inconvénients :

- **Complexité :** la synchronisation est obligatoire dès qu'il y a accès à des données partagées.
- **Débogage difficile :** les conditions de course (race conditions) et les interblocages (deadlocks) sont difficiles à reproduire.
- **Fragilité :** un bug dans un thread (ex. corruption mémoire) peut affecter tous les autres threads du processus.

[8] David R. Butenhof, "Programming with POSIX Threads", Addison-Wesley, 1997.

[10] The Open Group, "The Single UNIX Specification (POSIX.1-2017)", https://pubs.opengroup.org/onlinepubs/9699919799/

3.2 Créer un thread — pthread_create()

Synopsis

```
#include <pthread.h>
/* Compilation : gcc ... -lpthread */

int pthread_create(pthread_t        *idthread,
                   const pthread_attr_t *attr,
                   void *(*fonc)(void *),
                   void             *arg);

/* Retour : 0 = succès, code d'erreur POSIX sinon */
```

Le thread créé exécute la fonction fonc(arg). Son exécution commence dès que l'ordonnanceur lui attribue du temps CPU, ce qui peut être immédiatement ou après plusieurs commutations.

Exemple 1 — affichage concurrent

Code C

```
/* threads1.c */
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *affiche(void *arg) {
    char *lettre = (char *)arg;
    int i;
    for (i = 0; i < 6; i++) {
        printf("%s ", lettre);
        fflush(stdout);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    char a[] = "[A]", b[] = "[B]";

    pthread_create(&t1, NULL, affiche, a);
    pthread_create(&t2, NULL, affiche, b);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("\nFin.\n");
    return 0;
}
```

Résultat d'exécution

```
[A] [B] [A] [B] [B] [A] [A] [B] [A] [B] [A] [B]
Fin.
(L'entrelacement varie à chaque exécution)
```

3.3 Terminaison — pthread_exit() et pthread_join()

Synopsis

```
void pthread_exit(void *p_status);
/* Termine le thread appelant, transmet p_status au joignant */

int pthread_join(pthread_t thrd, void **code_retour);
/* Bloque jusqu'à la fin de thrd, récupère son code de retour */
/* Retour : 0 = succès */
```

Si le thread principal (main) se termine avant ses threads fils sans appeler pthread_join(), les threads fils sont tués abruptement. Il faut donc toujours synchroniser.

Exemple — thread retournant un calcul

Code C

```c
/* threads_join.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *somme_tableau(void *arg) {
    int *tab = (int *)arg;
    int n    = tab[0];    /* premier élément = taille */
    long *res = malloc(sizeof(long));
    *res = 0;
    int i;
    for (i = 1; i <= n; i++) *res += tab[i];
    pthread_exit(res);
}

int main(void) {
    pthread_t t;
    long *resultat;
    /* tab[0] = taille, tab[1..5] = valeurs */
    int tab[] = {5, 10, 20, 30, 40, 50};

    pthread_create(&t, NULL, somme_tableau, tab);
    pthread_join(t, (void **)&resultat);
    printf("Somme = %ld\n", *resultat);
    free(resultat);
    return 0;
}
```

Résultat d'exécution

```
Somme = 150
```

3.4 Détachement — pthread_detach()

Synopsis

```c
int pthread_detach(pthread_t t);
/* Un thread détaché libère ses ressources dès sa terminaison */
/* On ne peut plus appeler pthread_join() sur un thread détaché */
```

Un thread détaché est utile quand on n'a besoin ni de sa valeur de retour ni d'attendre sa terminaison explicitement. Le système libère ses ressources automatiquement à la fin.

3.5 Variables partagées et section critique

Les variables globales sont accessibles par tous les threads du processus. Une variable locale déclarée dans une fonction de thread est stockée sur la pile propre de ce thread et n'est donc pas accessible par les autres.

> **Définition — Condition de course (race condition) :** *Situation où le résultat d'un programme dépend de l'ordre d'entrelacement des instructions de plusieurs threads concurrents accédant à une ressource partagée.*

Exemple classique : deux threads incrémentent une variable partagée. L'opération n++ se décompose en trois instructions machine : lecture, incrément, écriture. Si l'ordonnanceur interrompt un thread entre la lecture et l'écriture, l'incrément de l'autre thread est perdu.

Code C

```c
/* race_condition.c — SANS protection, résultat incorrect */
#include <stdio.h>
#include <pthread.h>

#define N 1000000
int compteur = 0;   /* variable partagée */

void *incrementer(void *arg) {
    int i;
    for (i = 0; i < N; i++)
        compteur++;   /* NON atomique — race condition ! */
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, incrementer, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Compteur = %d (attendu %d)\n", compteur, 2*N);
    return 0;
}
```

Résultat d'exécution

```
Compteur = 1385241  (attendu 2000000)  ← résultat erroné !
```

Définition — Section critique : *Portion de code qui accède à une ressource partagée et qui ne doit être exécutée que par un seul thread à la fois.*

3.6 Exclusion mutuelle — mutex

Un mutex (Mutual EXclusion) est un verrou binaire : il est soit libre (unlocked), soit tenu par un thread (locked). Si un thread tente d'acquérir un mutex déjà tenu, il est bloqué (mis en attente passive) jusqu'à sa libération.

Fondements théoriques : les mutex sont une implémentation des sémaphores d'exclusion mutuelle (sémaphores binaires) introduits par Dijkstra[11].

Synopsis

```c
#include <pthread.h>

/* Initialisation statique (à la déclaration) */
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

/* Initialisation dynamique */
int pthread_mutex_init(pthread_mutex_t *m, const pthread_mutexattr_t *attr);

int pthread_mutex_lock(pthread_mutex_t *m);     /* bloquant    */
int pthread_mutex_trylock(pthread_mutex_t *m); /* non bloquant */
int pthread_mutex_unlock(pthread_mutex_t *m);  /* libérer     */
int pthread_mutex_destroy(pthread_mutex_t *m); /* détruire    */
```

Exemple — compteur protégé par mutex

Code C

```c
/* mutex_demo.c — version correcte */
#include <stdio.h>
#include <pthread.h>
```

[11] E.W. Dijkstra, "Cooperating Sequential Processes", Technical Report, Eindhoven University of Technology, 1965. (Introduction aux sémaphores.)

```c
#define N 1000000
int compteur = 0;
pthread_mutex_t verrou = PTHREAD_MUTEX_INITIALIZER;

void *incrementer(void *arg) {
    int i;
    for (i = 0; i < N; i++) {
        pthread_mutex_lock(&verrou);
        compteur++;            /* section critique protégée */
        pthread_mutex_unlock(&verrou);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementer, NULL);
    pthread_create(&t2, NULL, incrementer, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Compteur = %d (attendu %d)\n", compteur, 2*N);
    pthread_mutex_destroy(&verrou);
    return 0;
}
```

Résultat d'exécution

```
Compteur = 2000000 (attendu 2000000)  ← résultat correct
```

⚠ **Note :** Le mutex doit toujours être unlocked après la section critique, même en cas d'erreur — sous peine d'interblocage.

3.7 Variables de condition

Les variables de condition permettent à un thread d'attendre passivement qu'une condition logique devienne vraie, sans consommer de CPU (contrairement à l'attente active par boucle).

Référence : *Ben-Ari, « Principles of Concurrent and Distributed Programming »*[12]

Synopsis

```c
#include <pthread.h>

/* Initialisation statique */
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

/* Attendre la condition (libère le mutex pendant l'attente) */
int pthread_cond_wait(pthread_cond_t *cv, pthread_mutex_t *m);

/* Réveiller UN thread en attente */
int pthread_cond_signal(pthread_cond_t *cv);

/* Réveiller TOUS les threads en attente */
int pthread_cond_broadcast(pthread_cond_t *cv);

/* Détruire la condition */
int pthread_cond_destroy(pthread_cond_t *cv);
```

Patron d'utilisation (producteur / consommateur)

Code C

```c
/* cond_demo.c — producteur/consommateur */
#include <stdio.h>
```

[12] M. Ben-Ari, "Principles of Concurrent and Distributed Programming", 2nd ed., Addison-Wesley, 2006.

```c
#include <pthread.h>
#include <unistd.h>

/* Données partagées */
int donnee     = 0;
int produit    = 0;   /* booléen : donnée disponible ? */
pthread_mutex_t m  = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cv = PTHREAD_COND_INITIALIZER;

void *producteur(void *arg) {
    int i;
    for (i = 1; i <= 5; i++) {
        sleep(1);
        pthread_mutex_lock(&m);
        donnee  = i * 10;
        produit = 1;
        printf("Producteur : produit donnee=%d\n", donnee);
        pthread_cond_signal(&cv);     /* reveiller le conso */
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consommateur(void *arg) {
    int i;
    for (i = 0; i < 5; i++) {
        pthread_mutex_lock(&m);
        while (!produit)              /* attente passive */
            pthread_cond_wait(&cv, &m);
        printf("Consommateur : consomme donnee=%d\n", donnee);
        produit = 0;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

int main(void) {
    pthread_t tp, tc;
    pthread_create(&tc, NULL, consommateur, NULL);
    pthread_create(&tp, NULL, producteur,   NULL);
    pthread_join(tp, NULL);
    pthread_join(tc, NULL);
    return 0;
}
```

Résultat d'exécution

```
Producteur : produit donnee=10
Consommateur : consomme donnee=10
Producteur : produit donnee=20
Consommateur : consomme donnee=20
... (5 cycles au total)
```

⚠ **Note :** Utilisez TOUJOURS while (!condition) et non if (!condition) avec pthread_cond_wait() : les réveils spurieux (sans signal explicite) sont autorisés par POSIX.

Chapitre 4 — Gestion des signaux

Référence du chapitre : *Stevens & Rago, chap. 10* [13] *; Kerrisk, chap. 20–22* [14] *; man 7 signal*[15]

4.1 Notion de signal

> **Définition — Signal :** *Notification asynchrone envoyée à un processus pour l'informer qu'un événement s'est produit. Le signal interrompt le déroulement normal du processus et déclenche une action définie.*

Les signaux peuvent provenir de plusieurs sources :

- **Terminal :** combinaisons de touches : Ctrl+C → SIGINT, Ctrl+Z → SIGTSTP, Ctrl+\ → SIGQUIT.
- **Erreurs matérielles/logicielles :** violation de segmentation → SIGSEGV, division par zéro → SIGFPE, instruction illégale → SIGILL.
- **Appels système :** kill(), raise() (s'envoyer un signal à soi-même), alarm() (SIGALRM après un délai).
- **Noyau :** fin d'un fils → SIGCHLD, déconnexion du terminal → SIGHUP.

À la réception d'un signal, le processus peut adopter l'une de ces trois attitudes :

- **Action par défaut (SIG_DFL) :** définie par le système : terminaison, terminaison avec core dump, suspension, continuation, ou ignoré.
- **Ignorer (SIG_IGN) :** le signal est silencieusement ignoré.
- **Handler personnalisé :** une fonction utilisateur est exécutée à la réception du signal. Après son exécution, le processus reprend là où il avait été interrompu.

4.2 États d'un signal

- **Envoyé (sent) :** le signal a été émis en direction du processus.
- **Pending (en attente) :** le signal a été envoyé mais pas encore délivré (processus bloqué sur un appel système, signal masqué…).
- **Délivré :** le signal est pris en compte par le processus qui exécute l'action associée.

> ⚠ **Note :** Si le processus est en mode noyau (dans un appel système), le traitement du signal est différé jusqu'au retour en mode utilisateur.

4.3 Tableau des signaux courants

Signal	N°	Action par défaut	Description
SIGHUP	1	Terminaison	Déconnexion du terminal (hang up)
SIGINT	2	Terminaison	Interruption (Ctrl+C)

SIGQUIT	3	Core dump	Quitter (Ctrl+\)
SIGILL	4	Core dump	Instruction machine illégale
SIGFPE	8	Core dump	Erreur arithmétique (÷0, overflow)
SIGKILL	9	Terminaison forcée	Tuer — non interceptable
SIGSEGV	11	Core dump	Violation de segmentation mémoire
SIGPIPE	13	Terminaison	Écriture dans un tube sans lecteur
SIGALRM	14	Terminaison	Alarme temporelle (alarm())
SIGTERM	15	Terminaison	Terminaison (kill par défaut)
SIGUSR1	10	Terminaison	Signal utilisateur 1 (libre d'usage)
SIGUSR2	12	Terminaison	Signal utilisateur 2 (libre d'usage)
SIGCHLD	17	Ignoré	Enfant terminé ou stoppé
SIGSTOP	19	Suspension	Stopper — non interceptable
SIGCONT	18	Continuation	Reprendre après SIGSTOP
SIGTSTP	20	Suspension	Stopper (Ctrl+Z)

4.4 Envoyer un signal — kill()

Synopsis

```
#include <signal.h>
#include <sys/types.h>

int kill(pid_t pid, int sig);

/* Valeurs de pid :              */
/*  pid > 0  → processus pid     */
/*  pid = 0  → tout le groupe    */
/*  pid = -1 → tous processus    */
/*  pid < -1 → groupe |pid|      */
/*  sig = 0  → tester existence (aucun signal envoyé) */

/* Retour : 0 = succès, -1 = erreur */
```

Exemple 1 — le père envoie SIGUSR1 au fils

Code C

```
/* kill_demo.c */
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {           /* FILS */
        printf("Fils (PID=%d) : en attente...\n", getpid());
        while (1) sleep(1);  /* boucle infinie */
```

```
    }
    /* PERE */
    sleep(3);
    printf("Pere : envoi SIGUSR1 → fils PID=%d\n", pid);
    kill(pid, SIGUSR1);
    wait(NULL);
    printf("Pere : fils termine.\n");
    return 0;
}
```

Résultat d'exécution

```
Fils (PID=5302) : en attente...
Pere : envoi SIGUSR1 → fils PID=5302
Pere : fils termine.
```

Exemple 2 — tester l'existence d'un processus

Code C

```
/* exists.c */
#include <stdio.h>
#include <signal.h>
#include <errno.h>

int processus_existe(pid_t pid) {
    if (kill(pid, 0) == 0)  return 1;  /* existe */
    if (errno == EPERM)     return 1;  /* existe mais accès refusé */
    return 0;                           /* n'existe pas */
}

int main(void) {
    pid_t pids[] = {1, 99999};
    int i;
    for (i = 0; i < 2; i++)
        printf("PID %d : %s\n", pids[i],
               processus_existe(pids[i]) ? "existe" : "n'existe pas");
    return 0;
}
```

Résultat d'exécution

```
PID 1 : existe
PID 99999 : n'existe pas
```

4.5 Installer un gestionnaire — signal()

Synopsis

```
#include <signal.h>

typedef void (*sighandler_t)(int);
sighandler_t signal(int sig, sighandler_t handler);

/* handler peut être :                          */
/*   SIG_DFL  — restaurer le comportement par défaut */
/*   SIG_IGN  — ignorer le signal               */
/*   &ma_fonction — fonction handler utilisateur */

/* Retour : ancien handler, ou SIG_ERR en cas d'erreur */
```

Exemple 1 — intercepter Ctrl+C (SIGINT)

Code C

```
/* sigint_demo.c */
#include <stdio.h>
#include <signal.h>
```

```c
#include <unistd.h>

static int nb = 0;

void handler_sigint(int sig) {
    nb++;
    printf("\n[SIGINT recu %d fois]\n", nb);
    if (nb >= 2) {
        printf("Terminaison apres 2 Ctrl+C.\n");
        signal(SIGINT, SIG_DFL);  /* restaurer pour pouvoir tuer */
    } else {
        printf("Appuyez encore une fois pour quitter.\n");
    }
}

int main(void) {
    signal(SIGINT, handler_sigint);
    printf("Tapez Ctrl+C deux fois pour quitter.\n");
    while (1) {
        printf("."); fflush(stdout);
        sleep(1);
    }
    return 0;
}
```

Résultat d'exécution

```
Tapez Ctrl+C deux fois pour quitter.
...^C
[SIGINT recu 1 fois]
Appuyez encore une fois pour quitter.
...^C
[SIGINT recu 2 fois]
Terminaison apres 2 Ctrl+C.
```

Exemple 2 — handler pour SIGFPE

Code C

```c
/* fpe_demo.c */
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handler_fpe(int sig) {
    fprintf(stderr, "\n[ERREUR] Division par zero detectee !\n");
    exit(EXIT_FAILURE);
}

int main(void) {
    int a, b;
    signal(SIGFPE, handler_fpe);
    printf("Entrez a : "); scanf("%d", &a);
    printf("Entrez b : "); scanf("%d", &b);
    printf("%d / %d = %d\n", a, b, a / b);
    return 0;
}
```

Résultat d'exécution

```
Entrez a : 15
Entrez b : 0

[ERREUR] Division par zero detectee !
```

Exemple 3 — le fils ignore SIGUSR1

Code C

```c
/* ignore_signal.c */
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        signal(SIGUSR1, SIG_IGN);  /* ignorer SIGUSR1 */
        printf("Fils : SIGUSR1 est ignore.\n");
        while (1) sleep(1);
    }
    sleep(2);
    printf("Pere : envoi SIGUSR1...\n");
    kill(pid, SIGUSR1);            /* le fils l'ignore */
    sleep(2);
    printf("Pere : le fils est toujours en vie.\n");
    kill(pid, SIGTERM);            /* tuer avec SIGTERM */
    wait(NULL);
    return 0;
}
```

Résultat d'exécution

```
Fils : SIGUSR1 est ignore.
Pere : envoi SIGUSR1...
Pere : le fils est toujours en vie.
```

4.6 Signaux non interceptables

Trois signaux ont un statut particulier et ne peuvent être ni interceptés, ni bloqués, ni ignorés par un processus utilisateur :

- **SIGKILL (9) :** tue immédiatement le processus. Utilisé par kill -9 <pid>.
- **SIGSTOP (19) :** suspend le processus (l'état passe à stoppé).
- **SIGCONT (18) :** reprend l'exécution d'un processus stoppé.

Ces restrictions garantissent que le super-utilisateur (root) et le noyau peuvent toujours reprendre le contrôle d'un processus bloqué ou mal écrit.

Chapitre 5 — Communication inter-processus : les tubes

Référence du chapitre : *Stevens & Rago, chap. 15* [16] *; Kerrisk, chap. 44–46* [17] *; man 2 pipe, man 7 pipe, man 3 mkfifo*[18]

5.1 Tubes anonymes — pipe()

> **Définition — Tube anonyme (pipe) :** *Canal de communication unidirectionnel entre processus. Les données transitent en mode FIFO (First In, First Out) avec lecture destructive. Il n'existe que le temps de vie des processus qui l'utilisent.*

Un tube possède deux extrémités matérialisées par deux descripteurs de fichiers :

- **p[0] :** extrémité de lecture (sortie du tube).
- **p[1] :** extrémité d'écriture (entrée du tube).

Synopsis

```
#include <unistd.h>

int pipe(int p[2]);
/* Retour : 0 = succès, -1 = erreur */
```

Propriétés du tube anonyme :

- **Unidirectionnel :** les données vont de p[1] vers p[0].
- **FIFO :** les données sont lues dans l'ordre où elles ont été écrites.
- **Capacité limitée :** le tampon interne est typiquement de 64 Ko sous Linux (PIPE_BUF = 4096 octets pour les écritures atomiques).
- **Lecture bloquante :** si le tube est vide et qu'il a des écrivains, read() bloque jusqu'à l'arrivée de données.
- **EOF :** read() retourne 0 quand le tube est vide ET que tous les descripteurs d'écriture sont fermés.
- **SIGPIPE :** écrire dans un tube sans lecteur génère le signal SIGPIPE.

5.2 Écriture et lecture dans un tube

Synopsis

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t n);
/* Écrit n octets de buf dans fd. Retourne nb octets écrits. */

ssize_t read(int fd, void *buf, size_t n);
/* Lit jusqu'à n octets de fd dans buf.
   Retourne nb octets lus, 0 si EOF, -1 si erreur. */
```

Exemple 1 — communication père → fils

Code C

```c
/* pipe1.c */
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(void) {
    int p[2];
    char buf[128];
    const char *msg = "Donnees du pere au fils via le tube";

    if (pipe(p) == -1) { perror("pipe"); return 1; }

    pid_t pid = fork();
    if (pid == 0) {              /* FILS : lecteur */
        close(p[1]);             /* fermer l'écriture */
        ssize_t n = read(p[0], buf, sizeof(buf)-1);
        buf[n] = '\0';
        printf("Fils recu (%zd octets) : %s\n", n, buf);
        close(p[0]);
    } else {                     /* PERE : écrivain */
        close(p[0]);             /* fermer la lecture */
        write(p[1], msg, strlen(msg));
        close(p[1]);             /* EOF pour le fils */
        wait(NULL);
    }
    return 0;
}
```

Résultat d'exécution

```
Fils recu (35 octets) : Donnees du pere au fils via le tube
```

Exemple 2 — dialogue bidirectionnel (deux tubes)

Pour une communication bidirectionnelle, on utilise deux tubes : un père→fils et un fils→père.

Code C

```c
/* pipe2.c — deux tubes pour dialogue bidirectionnel */
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(void) {
    int p_pf[2], p_fp[2];  /* père→fils, fils→père */
    pipe(p_pf); pipe(p_fp);

    pid_t pid = fork();
    if (pid == 0) {          /* FILS */
        close(p_pf[1]); close(p_fp[0]);
        char q[64];
        read(p_pf[0], q, sizeof(q));
        printf("Fils recu : %s\n", q);
        char rep[] = "Bonjour Pere !";
        write(p_fp[1], rep, strlen(rep)+1);
        close(p_pf[0]); close(p_fp[1]);
    } else {                  /* PERE */
        close(p_pf[0]); close(p_fp[1]);
        char msg[] = "Bonjour Fils !";
        write(p_pf[1], msg, strlen(msg)+1);
        close(p_pf[1]);
        char rep[64];
        read(p_fp[0], rep, sizeof(rep));
```

```
        printf("Pere recu : %s\n", rep);
        close(p_fp[0]);
        wait(NULL);
    }
    return 0;
}
```

Résultat d'exécution

```
Fils recu : Bonjour Fils !
Pere recu : Bonjour Pere !
```

⚠ **Note :** Fermez toujours les extrémités non utilisées du tube (close(p[0]) chez l'écrivain, close(p[1]) chez le lecteur) pour éviter les interblocages.

5.3 Tubes nommés (FIFO) — mkfifo()

Définition — Tube nommé (FIFO) : *Fichier spécial dans le système de fichiers permettant à des processus sans lien de parenté d'échanger des données en mode FIFO. Il est visible par ls -l avec le type 'p'.*

Avantages par rapport au tube anonyme :

- **Sans parenté :** tout processus ayant les permissions appropriées peut l'utiliser.
- **Persistant :** il reste dans le système de fichiers après la fin des processus (jusqu'à unlink()).
- **Nommé :** identifiable par son chemin dans l'arborescence.

Création — mkfifo()

Synopsis

```
#include <sys/stat.h>

int mkfifo(const char *nom, mode_t mode);
/* Retour : 0 = succès, -1 = erreur (ex: fichier existe déjà) */

/* Exemples de mode : */
/* 0644 = rw-r--r-- (octal) */
/* S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH = identique */
```

5.4 Droits d'accès

Les droits s'expriment en octal sur 3 chiffres (propriétaire, groupe, autres). Chaque chiffre est la somme des permissions : 4=lecture, 2=écriture, 1=exécution.

Constante POSIX	Octal	Notation	Description
S_IRUSR	0400	r--------	Lecture propriétaire
S_IWUSR	0200	-w-------	Écriture propriétaire
S_IXUSR	0100	--x------	Exécution propriétaire
S_IRWXU	0700	rwx------	Tous droits propriétaire
S_IRGRP	0040	---r-----	Lecture groupe
S_IWGRP	0020	----w----	Écriture groupe
S_IROTH	0004	------r--	Lecture autres
S_IWOTH	0002	-------w-	Écriture autres

5.5 Ouverture, communication et suppression

Synopsis

```
#include <fcntl.h>

int open(const char *chemin, int flags);
/* flags : O_RDONLY (lecture) ou O_WRONLY (écriture) */
/* Ouverture bloquante par défaut :                  */
/*  O_RDONLY bloque jusqu'à qu'un écrivain ouvre     */
/*  O_WRONLY bloque jusqu'à qu'un lecteur ouvre      */
/* Ouverture non-bloquante : flags | O_NONBLOCK      */
```

Exemple complet — deux processus indépendants via FIFO

Fichier producteur.c :

Code C

```
/* producteur.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <sys/stat.h>

int main(void) {
    const char *fifo = "canal.fifo";
    int fd;
    int i;

    mkfifo(fifo, 0644);
    printf("Producteur : en attente d'un lecteur...\n");
    fd = open(fifo, O_WRONLY);  /* bloquant */

    for (i = 1; i <= 5; i++) {
        char msg[64];
        snprintf(msg, sizeof(msg), "Message numero %d\n", i);
        write(fd, msg, strlen(msg));
        printf("Producteur : envoye '%s'", msg);
        sleep(1);
    }
    close(fd);
    printf("Producteur : termine.\n");
    return 0;
}
```

Fichier consommateur.c :

Code C

```
/* consommateur.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    const char *fifo = "canal.fifo";
    int fd;
    char buf[128];
    ssize_t n;

    printf("Consommateur : ouverture du FIFO...\n");
    fd = open(fifo, O_RDONLY);  /* bloquant */

    while ((n = read(fd, buf, sizeof(buf)-1)) > 0) {
        buf[n] = '\0';
        printf("Consommateur recu : %s", buf);
    }
    close(fd);
```

```
    unlink(fifo);  /* supprimer le FIFO */
    printf("Consommateur : termine.\n");
    return 0;
}
```

Résultat d'exécution

```
# Terminal 1
$ ./producteur
Producteur : en attente d'un lecteur...
Producteur : envoye 'Message numero 1'
Producteur : envoye 'Message numero 2'
... (5 messages)
Producteur : termine.

# Terminal 2 (simultanément)
$ ./consommateur
Consommateur : ouverture du FIFO...
Consommateur recu : Message numero 1
Consommateur recu : Message numero 2
... (5 messages)
Consommateur : termine.
```

Compilation :

Code C

```
$ gcc -Wall -o producteur  producteur.c
$ gcc -Wall -o consommateur consommateur.c
```

Résumé des fonctions essentielles

Fonction	En-tête	Description
getpid/getppid/getuid	<unistd.h>	Identifiants du processus courant
fork()	<unistd.h>	Créer un processus fils (duplication)
exit()	<stdlib.h>	Terminer le processus avec code de retour
wait() / waitpid()	<sys/wait.h>	Attendre la fin d'un processus fils
execl/execv/execlp/execvp	<unistd.h>	Remplacer l'image du processus
sleep()	<unistd.h>	Suspendre pendant n secondes
errno	<errno.h>	Variable globale contenant le code d'erreur
perror() / strerror()	<stdio.h>/<string.h>	Afficher/retourner la description d'erreur
pthread_create	<pthread.h>	Créer un thread POSIX
pthread_join	<pthread.h>	Attendre la fin d'un thread
pthread_exit	<pthread.h>	Terminer le thread appelant
pthread_detach	<pthread.h>	Détacher un thread
pthread_mutex_lock/unlock	<pthread.h>	Verrouiller/libérer un mutex
pthread_mutex_trylock	<pthread.h>	Tenter verrouillage non bloquant
pthread_mutex_destroy	<pthread.h>	Détruire un mutex
pthread_cond_wait	<pthread.h>	Attente passive sur une condition
pthread_cond_signal	<pthread.h>	Réveiller un thread en attente
pthread_cond_broadcast	<pthread.h>	Réveiller tous les threads en attente
kill()	<signal.h>	Envoyer un signal à un processus
signal()	<signal.h>	Installer un handler de signal
raise()	<signal.h>	S'envoyer un signal à soi-même
alarm()	<unistd.h>	Programmer un SIGALRM
pipe()	<unistd.h>	Créer un tube anonyme
mkfifo()	<sys/stat.h>	Créer un tube nommé (FIFO)
open()	<fcntl.h>	Ouvrir un fichier ou un FIFO
read() / write()	<unistd.h>	Lire/écrire dans un descripteur
close()	<unistd.h>	Fermer un descripteur
unlink()	<unistd.h>	Supprimer un fichier/FIFO

Références bibliographiques

Ouvrages fondamentaux

[1] Brian W. Kernighan, Dennis M. Ritchie. The C Programming Language, 2ème éd. Prentice Hall, 1988. ISBN : 978-0131103627.

[2] W. Richard Stevens, Stephen A. Rago. Advanced Programming in the UNIX Environment, 3ème éd. Addison-Wesley, 2013. ISBN : 978-0321637734.

[3] Andrew S. Tanenbaum, Herbert Bos. Modern Operating Systems, 4ème éd. Pearson, 2014. ISBN : 978-0133591620.

[4] Michael Kerrisk. The Linux Programming Interface — A Linux and UNIX System Programming Handbook. No Starch Press, 2010. ISBN : 978-1593272203.

[5] David R. Butenhof. Programming with POSIX Threads. Addison-Wesley, 1997. ISBN : 978-0201633924.

[6] M. Ben-Ari. Principles of Concurrent and Distributed Programming, 2ème éd. Addison-Wesley, 2006. ISBN : 978-0321312846.

Articles et travaux de référence

[7] E.W. Dijkstra. Cooperating Sequential Processes. Technical Report EWD-123, Eindhoven University of Technology, 1965. (Introduction sémaphores.)

[8] Dennis M. Ritchie, Ken Thompson. The UNIX Time-Sharing System. Communications of the ACM, vol. 17, n° 7, pp. 365–375, 1974.

Ressources en ligne

[9] Documentation officielle du noyau Linux :
https://www.kernel.org/doc/html/latest/

[10] The Open Group — The Single UNIX Specification (POSIX.1-2017) :
https://pubs.opengroup.org/onlinepubs/9699919799/

[11] GNU C Library (glibc) Reference Manual :
https://www.gnu.org/software/libc/manual/

[12] Pages de manuel Linux (man pages) — en ligne :
https://man7.org/linux/man-pages/

[13] GCC Manual — Options de compilation :
https://gcc.gnu.org/onlinedocs/
[14] GDB — Débogueur GNU :
https://www.gnu.org/software/gdb/documentation/

Pr. Karima AISSAOUI — Université Mohammed Premier, Faculté des Sciences d'Oujda
k.aissaoui@ump.ac.ma — Polycopié de cours « Programmation Système en C sous Linux »